TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING

A

MINOR PROJECT MID-TERM REPORT

ON

CUSTOMER SEGMENTATION AND INTELLIGENT MOBILE RECOMMENDATION SYSTEM

SUBMITTED BY:

AAYUSH CHHUKA	(NCE080BCT002)
HOM RAJ BHANDARI	(NCE080BCT016)
SUDIP KUMAR TAMANG	(NCE080BCT042)
SURESH KHADKA	(NCE080BCT046)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

NATIONAL COLLEGE OF ENGINEERING

LALITPUR, NEPAL

JULY, 2026

Acknowledgements

We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering at National College of Engineering for providing us with the opportunity, facilities, and support needed to complete this minor project. The guidance and learning environment provided by the department helped us throughout the development of this project.

We would like to sincerely thank our project supervisor, Mr. Subash Pandey, for his continuous guidance, valuable suggestions, and patient support during every stage of the project. His feedback helped us improve our work and solve many technical and practical problems. His encouragement motivated us to complete the project with confidence.

Finally, we express our heartfelt thanks to our families, friends, and classmates for their constant encouragement, patience, and moral support throughout this journey. We are thankful to everyone who directly or indirectly contributed to the successful completion of this minor project.

Aayush Chhuka	NCE080BCT002
Hom Raj Bhandari	NCE080BCT016
Sudip Kumar Tamang	NCE080BCT042
Suresh Khadka	NCE080BCT046

Contents

Acknowledgements i

List of Figures iv

List of Tables v

List of Abbreviations vi

Chapter 1. Introduction 1

1.1 Background . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives . . . 2
1.3.1 General Objective . . . 2
1.3.2 Specific Objectives . . . 2
1.4 Scope and Limitations . . . 2
1.4.1 Scope . . . 2
1.4.2 Limitations . . . 3

Chapter 2. Literature Review 4

2.1 Related Work . . . 4
2.2 Related Theory . . . 4
2.2.1 Gradient Boosting Machines . . . 4
2.2.2 XGBoost Overview . . . 5
2.2.3 Mathematical Foundation . . . 5
2.2.4 Key Technical Features . . . 6
2.2.5 Why XGBoost for This Project? . . . 7
2.3 Website Development for End Users . . . 7
2.3.1 User Interface Design . . . 7
2.3.2 Customer Functionality . . . 7
2.3.3 Salesman Functionality . . . 8
2.3.4 Admin Functionality . . . 8

Chapter 3. Proposed Methodology 9

3.1 Software Development Model 9
3.2 Feasibility Study 10
3.2.1 Technical Feasibility 10
3.2.2 Economic Feasibility 11
3.2.3 Operational Feasibility 11
3.2.4 Legal and Ethical Feasibility 12
3.3 Requirement Analysis 12
3.3.1 Functional Requirements 12
3.3.2 Non Functional Requirements 14
3.4 Proposed System Design 15
3.4.1 System Architecture 15
3.4.2 Use Case Diagram 16
3.4.3 Data Flow Diagram 17
3.4.4 System Flowchart 19
3.4.5 Data Preparation and Machine Learning Workflow 20
3.5 Implementation Details 22
3.5.1 Frontend Development 22
3.5.2 Backend Development 22
3.5.3 Machine Learning Model Integration 23
3.5.4 Database Schema 23
3.5.5 Testing Strategy 23

Chapter 4. Time Schedule **25**

Chapter 5. Expected Output **26**
5.1 Work Completed 26
5.1.1 Frontend 26
5.1.2 Backend 26
5.1.3 Machine Learning 26
5.2 Work in Progress 27
5.2.1 Frontend 27
5.2.2 Backend 27
5.2.3 Machine Learning 27

References **28**

List of Figures

I Iterative Software Development Model used in this project 9
II System Architecture of the Mobile Recommendation System 15
III Use Case Diagram of the System 16
IV Level 0 Data Flow Diagram of the System 17
V Level 1 Data Flow Diagram of the System 18
VI System Flowchart Showing Admin, Salesman, and Customer Workflows 19
VII Data Collection and Integration Process 20
VIII Machine Learning Model Workflow 21

I Gantt chart for Customer Segmentation and intelligent Mobile Recommendation System 25

List of Tables

I Economic Feasibility Estimation 11

List of Abbreviations

API	Application Programming Interface
BCT	Bachelor in Computer Engineering
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheet
CSV	Comma Separated Values
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
ID	Identifier
JSON	JavaScript Object Notation
ML	Machine Learning
ORM	Object Relational Mapper
OTP	One Time Password
RAM	Random Access Memory
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
UI	User Interface
XGBoost	Extreme Gradient Boosting

1. Introduction

1.1 Background

The mobile phone market offers a large number of brands and models with different prices, features, and specifications. Because of this wide variety, customers often find it difficult to choose a mobile phone that matches their needs and budget. At the same time, mobile retailers face challenges in understanding customer preferences and recommending suitable products. As a result, customers may spend more time searching for a phone, and retailers may miss opportunities to provide better recommendations [1].

Customer segmentation is a technique that divides customers into groups based on similar characteristics and buying behavior. Instead of treating every customer the same, businesses can identify groups with similar interests and provide recommendations that better match their needs [1].

In the mobile phone market, customer preferences are influenced by many factors. Some customers look for high performance devices for gaming or professional work, while others focus on battery life, camera quality, display, storage, or overall value for money. Many customers also have strong preferences for specific brands because of their previous experience, trust, or familiarity with the brand.

Many existing recommendation systems depend on a customer's previous purchases, browsing history, or ratings to recommend products. This creates a problem, known as the cold-start problem, for new customers who have little or no interaction history [2]. It also becomes difficult to recommend newly released mobile phones because there is limited historical data available for them. In addition, many mobile retailers still depend on manual analysis or simple sales reports to understand customer groups, which may not accurately capture changing customer preferences.

This report presents the progress completed for the mid term defense. At this stage, the data collection and preprocessing tasks have been completed. The backend services have been developed using Node.js, and the first version of the web application interface has also been implemented. The machine learning model training, model integration, recommendation engine, and complete system evaluation will be carried out in the next phase of the project.

1.2 Problem Statement

The mobile phone market offers a wide range of smartphones with different prices, features, and specifications. This makes it difficult for retailers to understand customer

preferences and recommend the most suitable mobile phones. Customer preferences are influenced by factors such as performance requirements, camera quality, battery life, budget, and brand preference, which cannot be accurately identified using basic demographic information alone.

Retailers also face challenges in grouping customers based on their purchasing behavior and using this information to support product recommendations and marketing decisions. Without an intelligent customer segmentation system, recommendations and promotional activities may not effectively match the interests of different customer groups.

Therefore, there is a need to develop a machine learning based customer segmentation system that can analyze customer and product data, identify meaningful customer groups, and support intelligent mobile phone recommendations.

1.3 Objectives

1.3.1 General Objective

To develop an intelligent mobile recommendation system using machine learning based customer segmentation that recommends suitable smartphones based on customer preferences and purchasing behavior.

1.3.2 Specific Objectives

I. To collect and preprocess mobile phone data from GSMArena and AnTuTu benchmark through web scraping.

II. To develop a secure web application with user authentication and database management.

III. To build a machine learning model for customer segmentation.

IV. To develop a recommendation system based on the identified customer segments.

V. To integrate the machine learning model with the web application.

VI. To evaluate the performance and accuracy of the proposed system.

1.4 Scope and Limitations

1.4.1 Scope

The project includes the following scope:

I. The project uses mobile phone specifications collected from GSMArena and benchmark scores collected from NanoReview.

II. The project develops a machine learning based customer segmentation model for intelligent mobile phone recommendations.

III. The project includes a web based application with user authentication, database management, and recommendation features.

IV. The project integrates customer data and mobile phone specifications to provide personalized recommendations.

1.4.2 Limitations

The project has the following limitations:

I. The system is limited to a web application and does not include a mobile application.

II. The system does not connect with live e commerce platforms or online payment systems.

III. The system uses only the data collected during the project.

IV. The recommendation performance depends on the quality and quantity of the available data.

2. Literature Review

2.1 Related Work

Chen and Guestrin [4] introduced XGBoost, a scalable tree boosting system that has become a benchmark for tabular data. Its regularized learning objective and efficient handling of mixed features make it ideal for customer segmentation tasks, directly supporting its use as the sole algorithm in this project.

Singh [1] developed a customer segmentation and recommendation system with a similar multi-dimensional approach, categorizing users into segments like Tech Savvy, Brand Lovers, and Camera Lovers based on purchase behavior. Their work, publicly available on GitHub, demonstrates a practical application of micro-segmentation for mobile phone recommendations using machine learning and a Flask backend. This project serves as a key reference for both the segmentation logic and the overall system architecture.

Qi et al. [5] proposed an XGBoost recommendation algorithm integrated with collaborative filtering to specifically address the cold-start problem. By using XGBoost to predict item preferences for new users, their work validates XGBoost's capability to generate recommendations without historical purchase data, a core requirement for this project's customer recommendation feature.

Ayyappan et al. [6] demonstrated a hybrid BiLSTM-XGBoost model for purchase intent prediction. While their work shows XGBoost's effectiveness as a final classifier, this project focuses on a pure XGBoost approach for all segmentation and regression tasks, proving its standalone power for structured, tabular phone specifications and transaction data.

2.2 Related Theory

2.2.1 Gradient Boosting Machines

Gradient Boosting is a machine learning technique that builds a strong predictive model by combining many simple models in a sequential manner. The core idea is to train new models that correct the mistakes made by previously trained models. Each new model focuses on the difficult cases that earlier models could not handle correctly, gradually improving the overall prediction accuracy.

The process begins with a simple initial prediction, often just the average value of the target variable. In each subsequent step, a new model is trained on the errors or residuals of the combined previous models. These residuals represent the gap between the actual

values and the predictions made so far. By learning to predict these residuals, each new model helps close this gap, pushing the ensemble toward better performance.

2.2.2 XGBoost Overview

XGBoost, which stands for Extreme Gradient Boosting, is an advanced implementation of gradient boosting that has become one of the most popular and successful machine learning algorithms. It was designed with both speed and accuracy in mind, making it suitable for a wide range of prediction tasks.

The algorithm builds an ensemble of decision trees one after another, where each tree learns from the mistakes of all previous trees. Unlike a random forest where trees are built independently, the trees in XGBoost work together as a team, with each new member focusing on what the team got wrong. This sequential learning approach allows the model to capture complex patterns in the data that simpler models might miss.

2.2.3 Mathematical Foundation

The learning process in XGBoost is guided by a carefully designed objective function that balances two competing goals. The first goal is to make accurate predictions, measured by a loss function that quantifies the difference between predicted and actual values. The second goal is to keep the model simple and general, measured by a regularization term that penalizes overly complex trees.

The overall objective that XGBoost minimizes during training can be expressed as:

$$\mathcal{L}(\theta) = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k) \tag{2.1}$$

where:

- $\mathcal{L}(\theta)$ is the overall objective function that XGBoost minimizes during training. It represents the total cost that balances prediction errors against model complexity.
- n is the total number of training samples in the dataset.
- y_i is the actual true value for the i-th training sample.
- $\hat{y}_i$ is the predicted value produced by the model for the i-th training sample.
- $l(y_i, \hat{y}_i)$ is the loss function that measures the difference between the actual value and the predicted value for a single sample. For regression tasks this is often squared error, and for classification tasks it is often logarithmic loss.
- K is the total number of decision trees in the ensemble.
- f_k represents the k-th decision tree in the ensemble.

- $\Omega(f_k)$ is the regularization term that penalizes the complexity of the k-th tree to prevent overfitting.

In this equation, the first term sums the loss across all n training samples, where l measures how wrong the prediction is for each sample. The loss function can be chosen based on the task, such as squared error for regression problems or logistic loss for classification. The second term adds up the complexity penalty for all K trees in the ensemble. This penalty term takes the form:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 \tag{2.2}$$

Here T represents the number of leaves in the tree, and w represents the scores assigned to each leaf. The gamma parameter controls how much the model is penalized for adding more leaves, helping to prevent the tree from growing too deep and memorizing noise in the data. The lambda parameter controls the magnitude of the leaf scores, encouraging the model to make more conservative predictions.

2.2.4 Key Technical Features

XGBoost introduces several innovations that contribute to its strong performance. One important feature is the use of second order derivatives during tree construction. While basic gradient boosting only uses the first derivative of the loss function, XGBoost also uses the second derivative, which provides more information about the shape of the loss surface. This allows the algorithm to make better decisions about how to split the data at each tree node.

Another crucial feature is the built in handling of missing values. In real world datasets, missing values are common and can be problematic for many algorithms. XGBoost learns the best direction to send missing values during training, effectively letting the data tell the algorithm how to handle gaps. This eliminates the need for separate imputation steps and often leads to better models.

Regularization is deeply integrated into XGBoost, going beyond simple pruning of trees after they are built. Both L1 and L2 regularization are applied directly to the leaf scores, which helps prevent individual trees from becoming too confident in their predictions. This regularization, combined with techniques like column subsampling where each tree only sees a random subset of features, helps the ensemble generalize well to new data.

2.2.5 Why XGBoost for This Project?

For this project, XGBoost handles the mixed data types present in our phone specifications and transaction records without requiring extensive preprocessing. The algorithm is robust to outliers in features like price and benchmark scores, which might otherwise skew the results. Its built in regularization helps ensure that our models will perform well on new customers and phones they have not seen during training.

The sequential learning nature of gradient boosting means XGBoost can focus on the difficult cases in our data. In customer segmentation, some customers may have clear preferences that are easy to predict, while others have more complex behavior patterns. XGBoost will naturally allocate more of its capacity to learning these challenging cases, potentially improving the quality of recommendations for all customers.

Finally, XGBoost has a proven track record in both academic research and industry applications involving tabular data. Its consistent performance across many different types of problems makes it a reliable choice for implementing all the models in our segmentation pipeline, from tech tier classification to brand loyalty prediction.

2.3 Website Development for End Users

2.3.1 User Interface Design

The website has a clean and simple interface made for three kinds of users. These are customers who want to buy a phone, salesmen who work in mobile shops, and admins who manage the system. The home page shows clear links for each group so anyone can find what they need without confusion. The design focuses on ease of use. Pages load fast, buttons are easy to find, and the layout works well on both computers and phones. Users do not need any technical knowledge to use the site.

2.3.2 Customer Functionality

A customer can sign up as a new user or log in if they already have an account. The sign up form asks for a full name, an email address, a password, and a password confirmation. After the form is filled, a six digit code is sent to the email. The customer enters this code to confirm the account. The React side of the app already supports this multi step sign up. It also supports the code entry box where the code can be pasted, a timer that tells when a new code can be sent, and the option to reset a forgotten password. Customers who already have an account can just enter their email and password to log in. Once the recommendation part is added in the next phase, the site will show a list of phones that match what the customer is looking for.

2.3.3 Salesman Functionality

A salesman logs in and lands on a separate dashboard. The main tool on this dashboard is a search box where the salesman types a city or area name. After the search, the site shows the phone brands and models that sell well in that area. The results include the full list of phone details, the camera quality group, and the performance group. With this view, the salesman can decide which phones to keep in stock and which customers to talk to about them. At the mid term defense, a simple version of this dashboard has been built in React. It shows the layout and the type of information that will appear. The real prediction features will be added in the next phase.

2.3.4 Admin Functionality

An admin is the person who looks after the whole system. The admin can see all the users in the database. The admin can add a new user, change a user's details, or remove a user when needed. The admin can also assign a role to any user, such as customer, salesman, or admin. These role assignment tasks are done through special pages that only admins can open. The admin page is part of the next phase of work. The backend already has the routes and the access checks for these tasks, so the front end can be built on top of what is in place.

3. Proposed Methodology

3.1 Software Development Model

The project is developed using the **Iterative Model** with parallel development across team members. This model was chosen because the work is split into frontend, backend, and machine learning parts that are built at the same time, with each part going through repeated cycles of design, build, and test until the full system is ready.

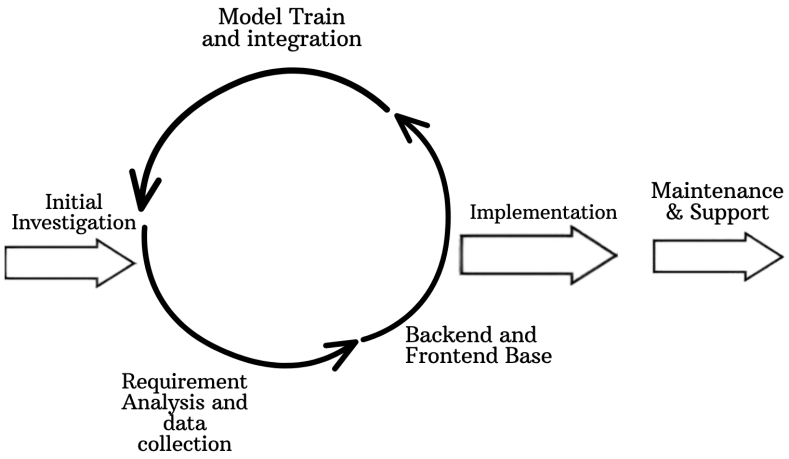

Figure I: Iterative Software Development Model used in this project

The Iterative Model is suitable for this project because it allows the system to be developed in small stages with continuous improvements. The project consists of frontend, backend, database, and machine learning components, which can be developed and tested independently while gradually integrating them into a complete system. This approach allows the team to identify and fix issues early, add new features in each iteration, and improve the system based on testing results and feedback from the project supervisor. It also provides flexibility to refine both the web application and the machine learning model throughout the development process.

The project is divided into four short cycles. Each cycle adds a new layer of the system on top of what already works.

I. **Iteration 1: Requirement Analysis and Data Collection.** The team gathered the project requirements, selected the datasets, and prepared the merged dataset. This was completed before development began.

II. **Iteration 2: Backend and Frontend Base.** The Node.js backend with Express

and the React frontend were built in parallel. The authentication flow, database setup, and basic pages were finished.

III. **Iteration 3: Machine Learning Training and Model Integration.** The XGBoost models will be trained on the merged dataset, and a small Python service will be built to serve predictions to the Node.js backend.

IV. **Iteration 4: Recommendation Features and Testing.** The recommendation endpoints, the customer and salesman dashboards, and the final testing will be completed in this cycle.

3.2 Feasibility Study

The feasibility study evaluates whether the proposed customer segmentation and mobile recommendation system can be realistically developed and deployed under existing constraints.

The project flow follows standard SDLC phases from planning and analysis through implementation, testing, and deployment.

3.2.1 Technical Feasibility

The proposed system is technically feasible given the team's proficiency and access to essential tools and technologies.

Technology Stack:

- Frontend: React based single page application using HTML, CSS, and JavaScript, with Vite compatible tooling through Create React App, Axios style fetch calls, and Bootstrap friendly component structure for responsive layout.
- Backend: Node.js with the Express.js framework for server side logic, including routing, session management, and middleware based authentication.
- Database: PostgreSQL accessed through the Prisma ORM, providing typed schema definitions, automatic migrations, and referential integrity for users and OTP records.
- Machine Learning: Python with XGBoost, pandas, and scikit-learn for the data preprocessing and segmentation models. The trained models will be served to the Node.js backend through a small HTTP bridge.
- Version Control: Git and GitHub for collaborative development and code management.
- Runtime: Node.js for the backend, npm for package management, nodemon for development, and the React development server for the frontend.

Team Skills: The development team has foundational skills in JavaScript and TypeScript for the application layer, basic Python for data work, SQL and relational database concepts, and introductory machine learning. The team has experience with REST APIs, JSON based request response flows, and modern frontend component design.

Development Tools: VS Code for code editing, Jupyter Notebook for exploratory data analysis, Git and GitHub for version control, Postman for API testing, pgAdmin for the PostgreSQL database, and the nodemon development server for the Node.js backend.

3.2.2 Economic Feasibility

The proposed system is economically feasible as it relies entirely on open source tools and free development platforms. The Node.js runtime, Express.js framework, Prisma ORM, PostgreSQL database, React library, XGBoost library, pandas, and scikit-learn are all freely available with no licensing costs.

Cost assumptions are based on academic project requirements and free tier boundaries of selected services.

Table I: Economic Feasibility Estimation

Item	**Estimated Cost**
Development Tools	Rs. 0 (open source)
JavaScript Libraries	Rs. 0 (React, Express, Prisma all open source)
Python Libraries	Rs. 0 (XGBoost, pandas, numpy, scikit-learn all open source)
Machine Learning Models	Rs. 0 (no licensing costs for model development)
Dataset Acquisition	Rs. 0 (Web Scraping)
Domain Name	Rs. 0 (free for academic demonstration)
Web Hosting	Rs. 0 (local hosting or free tier cloud)
Miscellaneous	Rs. 500-1,000
Total Initial Cost	**Rs. 500-1,000**

For academic demonstration purposes, local hosting on development servers will suffice. Commercial deployment would require cloud hosting and domain registration expenses as the user base grows.

3.2.3 Operational Feasibility

The system is operationally feasible as it aligns well with the team's development capabilities and workflow. The web based platform can be accessed through any modern browser on desktop and mobile devices, eliminating the need for platform specific development. The interface is designed to be simple for the three groups of users. Cus-

tomers look for phones that match their needs. Salesmen look for market insights and stock decisions. Admins look after the system and the users in it.

The platform requires little training for any user. Salesmen can enter a location and see which phones are popular in that area. Customers can register, set their preferences, and get recommendations with only a few clicks. Admins can view all users, add or remove users, and assign roles through simple pages. The clear three pathway design makes the system easy to use for both technical and non technical users.

3.2.4 Legal and Ethical Feasibility

I. The phone specifications used in this project are collected through web scraping from GSMArena, a public website that lists mobile phone details. Only the specifications that GSMArena makes publicly visible are collected.

II. The AnTuTu benchmark scores are collected through web scraping, which publishes these scores for public use. The scores are used for academic and educational purposes only.

III. The data collected does not contain any personal user information. There are no names, email addresses, or contact details of any individual in the scraped data.

IV. As an academic project, the system is not built for commercial use. It will be used only for learning, demonstration, and evaluation within the project.

V. The project follows general ethical practices for machine learning work. The data sources are credited properly, and the system is not used in any way that can harm users or other parties.

3.3 Requirement Analysis

3.3.1 Functional Requirements

The functional requirements describe what the system must do. They are grouped by user role.

Customer Functions:

- A new user must be able to register by providing a full name, an email address, a password, and a password confirmation.
- A new user must receive a six digit code on the email after registration and must enter this code to confirm the account.
- A registered user must be able to log in using email and password.
- A user must be able to reset the password by requesting a code on the registered email.

- A user must be able to set and update personal preferences such as camera quality, performance, and budget.
- A user must be able to view a list of recommended phones based on the set preferences.
- A user must be able to view the details of each recommended phone.

Salesman Functions:

- A salesman must be able to log in to a separate dashboard using email and password.
- A salesman must be able to enter a city or area name and view the phones that are popular in that area.
- A salesman must be able to view phone specifications, camera category, and performance category for each result.
- A salesman must be able to use the insights to decide which phones to keep in stock.

Admin Functions:

- An admin must be able to view the list of all users in the system.
- An admin must be able to add a new user, update user details, or remove a user.
- An admin must be able to assign or change the role of any user, such as customer, salesman, or admin.
- Only admins must be able to access the user management pages.

System Functions:

- The system must classify users by tech level, camera preference, regional brand popularity, and brand loyalty behavior.
- The system must recommend phones to customers based on the trained machine learning models.
- The system must predict the most suitable audience for newly launched phones by combining multiple model outputs.
- The system must store user details, phone specifications, and recommendation history in the database.

3.3.2 Non Functional Requirements

The non functional requirements describe how the system performs rather than what it does.

- **Performance.** The system should return prediction results in under two seconds. Pages should load fast and respond quickly to user actions.
- **Usability.** The interface should be simple and clear. Users should be able to find what they need with only a few clicks, and the layout should work well on both desktop and mobile screens.
- **Reliability.** The system should remain available throughout the day. It should handle temporary backend failures without crashing and should show proper error messages when something goes wrong.
- **Security.** Passwords must be stored in hashed form. One time codes must expire after a fixed time. User sessions must be managed safely, and protected routes must reject users who are not logged in.
- **Scalability.** The system should support future growth in users, phones, and data. New machine learning models should be added without major changes to the existing code.
- **Compatibility.** The website should work on all major browsers and on different screen sizes, including smaller mobile screens.
- **Maintainability.** The codebase should be split into clear modules with proper documentation, coding standards, and version control. Models should be retrained independently of the application code.
- **Availability.** The system should keep working on slow internet connections without losing the user session or breaking the page.

3.4 Proposed System Design

3.4.1 System Architecture

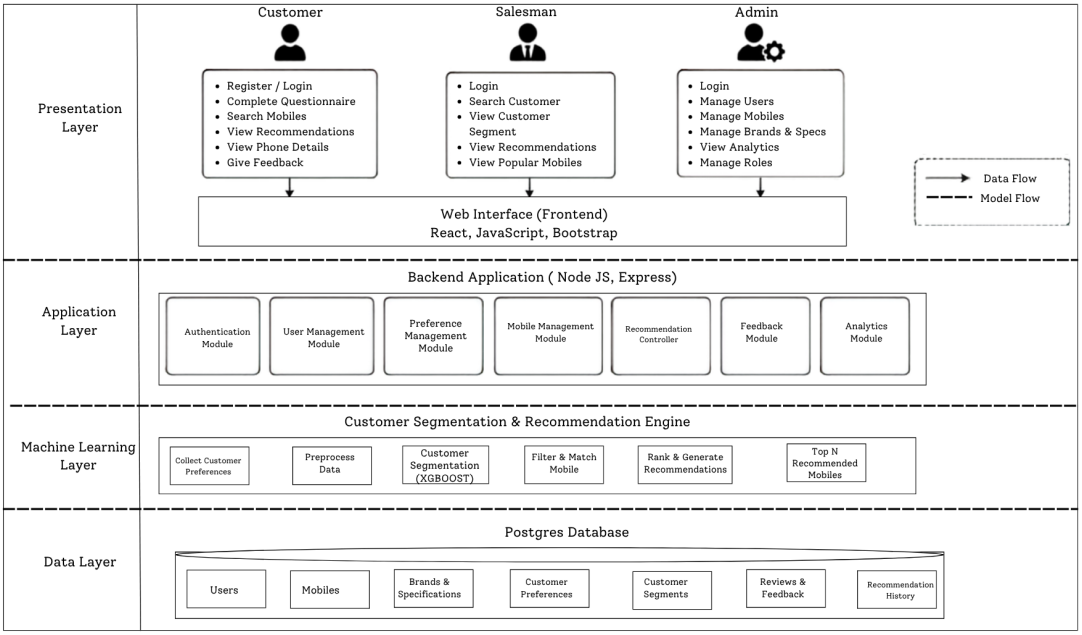

Figure II: System Architecture of the Mobile Recommendation System

The system follows a modular three tier architecture with clear separation between the data, model, and application layers, and the overview is shown in Figure II. The frontend is a React based single page application that talks to the backend over JSON HTTP. The Node.js backend, built on the Express.js framework, handles HTTP routing, user authentication, session management, and will orchestrate calls to the machine learning models in the next project phase. The model layer consists of XGBoost models trained in Python that perform the core segmentation and recommendation tasks. PostgreSQL, accessed through the Prisma ORM, serves as the relational database for storing user profiles, OTP records, and future tables for phone specifications, customer preferences, and recommendation history. The architecture ensures that each component can be developed, tested, and maintained independently while working together through well defined APIs.

3.4.2 Use Case Diagram

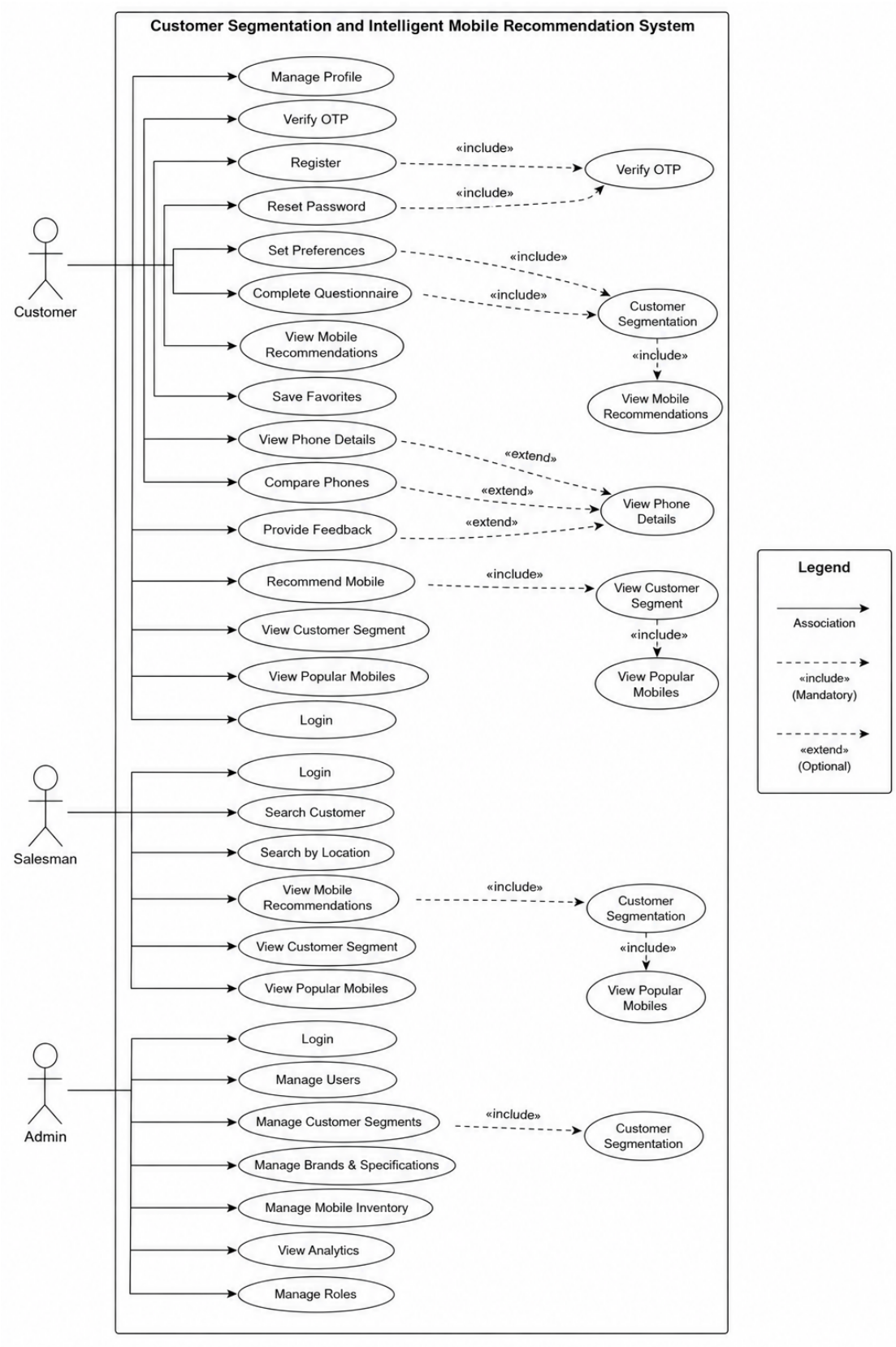

Figure III: Use Case Diagram of the System

The use case diagram in Figure III shows how the three user groups, which are the customer, the salesman, and the admin, interact with the system. Each actor is connected to the actions they can perform, and shared actions like login are shown once and connected to all the actors that need them. The customer can register, verify the OTP, log in, set preferences, reset the password, and view phone recommendations. The salesman can log in, search by location, and view the popular phones along with their camera and performance categories. The admin can log in, manage users, and assign roles. The system itself handles shared tasks like authentication, storing data, and running the machine learning models that power the recommendations.

3.4.3 Data Flow Diagram

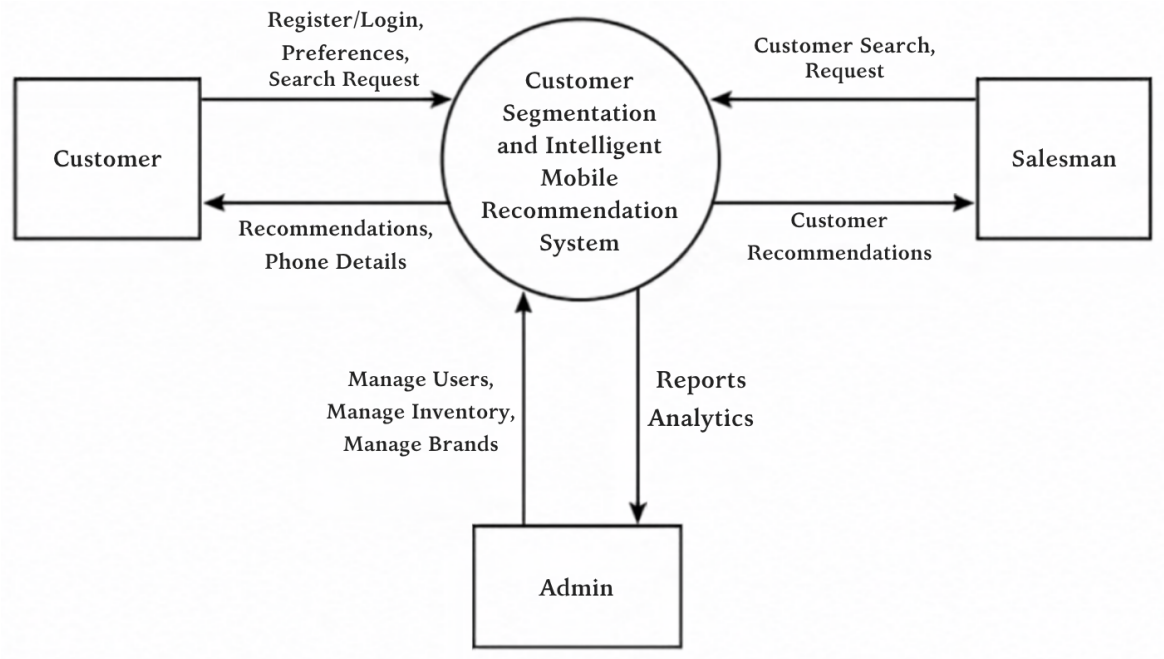

Figure IV: Level 0 Data Flow Diagram of the System

The Level 0 data flow diagram in Figure IV gives the high level view of the three external entities, which are the customer, the salesman, and the admin, sending requests to the system and receiving responses.

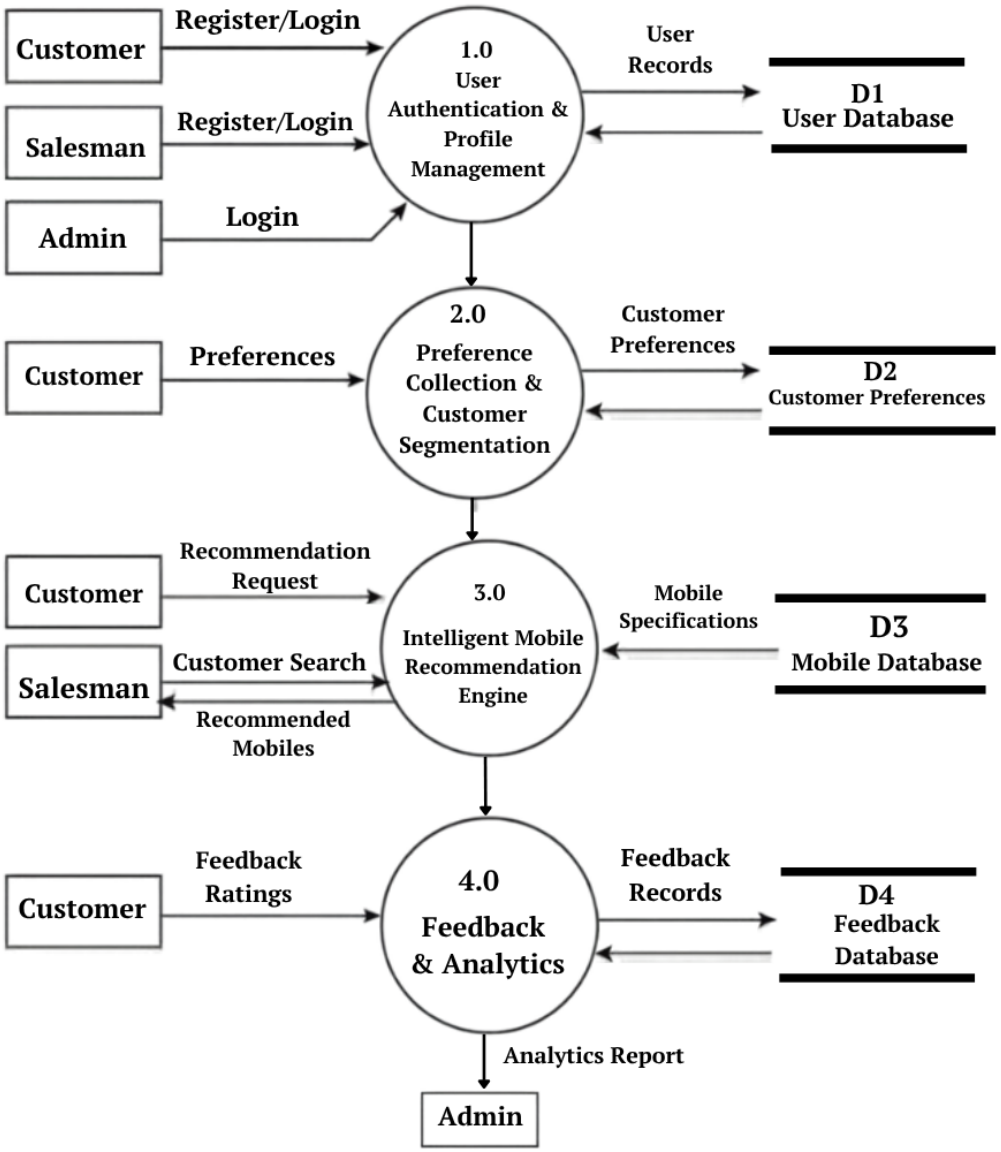

Figure V: Level 1 Data Flow Diagram of the System

The Level 1 data flow diagram in Figure V breaks the system into its main processes, which are the React frontend, the Node.js backend, the Python XGBoost service, and the PostgreSQL database, and shows how data flows between them. The customer, salesman, and admin send their requests through the frontend, which passes them to the Node.js backend. The backend checks the request, talks to the PostgreSQL database to read or write user data, and when a prediction is needed it calls the Python XGBoost service. The model returns the prediction, the backend combines it with data from the database, and the final response is sent back to the frontend for the user to see. This way each part of the system only handles the data it is responsible for, and the flow stays clear from start to finish.

3.4.4 System Flowchart

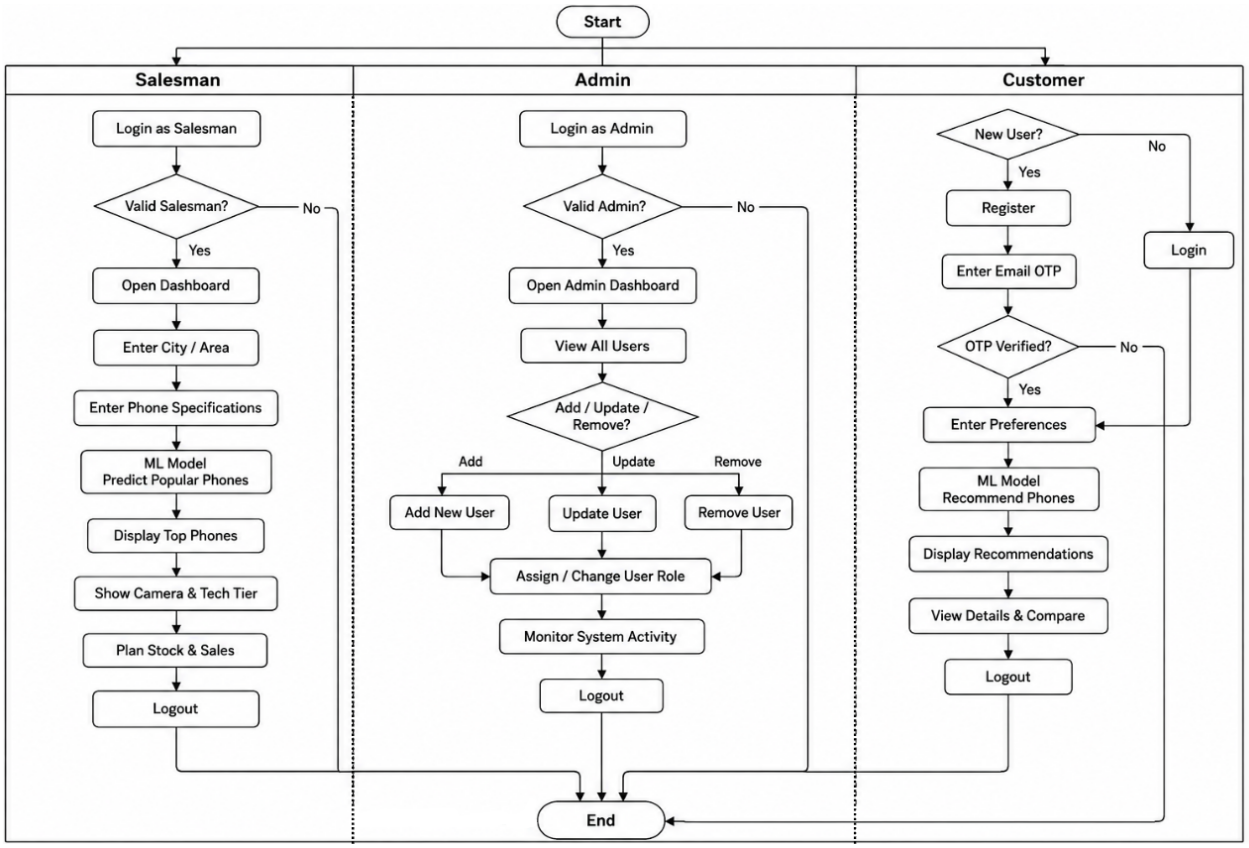

Figure VI: System Flowchart Showing Admin, Salesman, and Customer Workflows

The system flowchart shows the complete path that each user group follows from the moment they enter the system to the moment they leave. The flow starts at the top with a single start node, and then branches into three separate lanes for three different roles. One lane is for the admin, one lane is for the salesman, and one lane is for the customer.

Admin Lane. The admin begins by entering the login screen. The system checks whether the login is valid. If the login is correct, the admin opens the admin dashboard and views the list of all users in the system. The admin then chooses one of three actions: add a new user, update an existing user, or remove a user from the system. After the action is done, the flow comes back to a single step where the admin can assign or change the role of any user. Once the user management work is done, the admin can move to the system monitoring step to check the overall activity in the system. Finally, the admin logs out and the flow reaches the end node. If the login is not valid, the flow also reaches the end node.

Salesman Lane. The salesman begins by entering the login screen. The system checks whether the login is valid. If the login is correct, the salesman opens the salesman dashboard. The salesman then enters required details. This input is passed to the machine

learning model, which predicts the suitable phone. The system displays the top phones along with their camera tier and tech tier details. The salesman uses these results to plan which phones to keep in stock and which customers to target. Finally, the salesman logs out and the flow reaches the end node. If the login is not valid, the flow also reaches the end node.

Customer Lane. The customer is first asked whether they are a new user or a returning user. New users are sent to the registration step, where they enter their name, email, and password. A six digit code is then sent to their email, and the customer enters this code in the system. The system checks whether the code is correct. If the code is verified, the customer moves to the preference step. If the code is not correct, the flow reaches the end node. Returning users skip the registration step and go straight to the login step. From the preference step onwards, both new and returning customers enter their preferences. The machine learning model then recommends matching phones, and the customer views the recommended phones along with their full details. The customer can compare the options and then log out. After the logout, the flow reaches the end node.

3.4.5 Data Preparation and Machine Learning Workflow

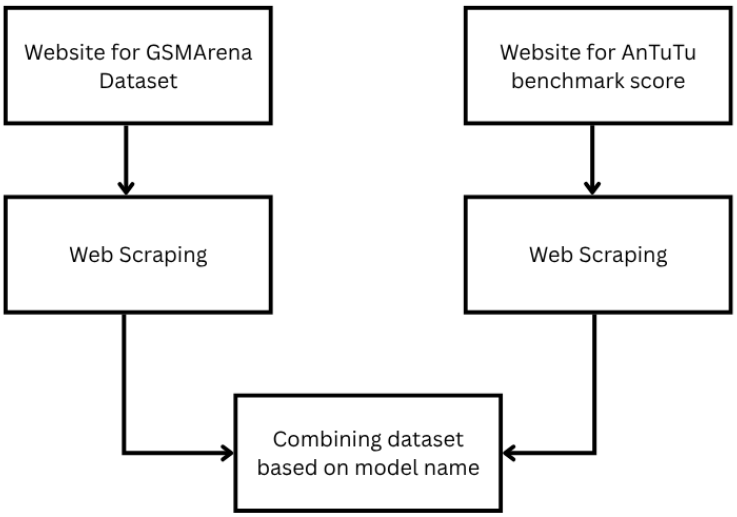

Figure VII: Data Collection and Integration Process

The data collection process begins with two primary sources. Phone specifications are obtained through web scraping from the GSMArena website. The full list of phones on GSMArena is divided into small batches, and each batch is scraped one at a time to avoid overloading the website and to keep the scraping process stable. The scraped data

includes the full hardware details of each phone such as the processor, RAM, storage, and camera information. AnTuTu benchmark scores are also collected through web scraping from the AnTuTu website, which results in a CSV file containing performance scores for different phone models. These two sources are merged into a single dataset based on the model name, so that each phone in the final dataset has both its hardware specifications and its benchmark score. This data preparation step was completed in the first week of the project before the development phase began. The merged dataset will serve as the foundation for all the model training and analysis work in the next phase.

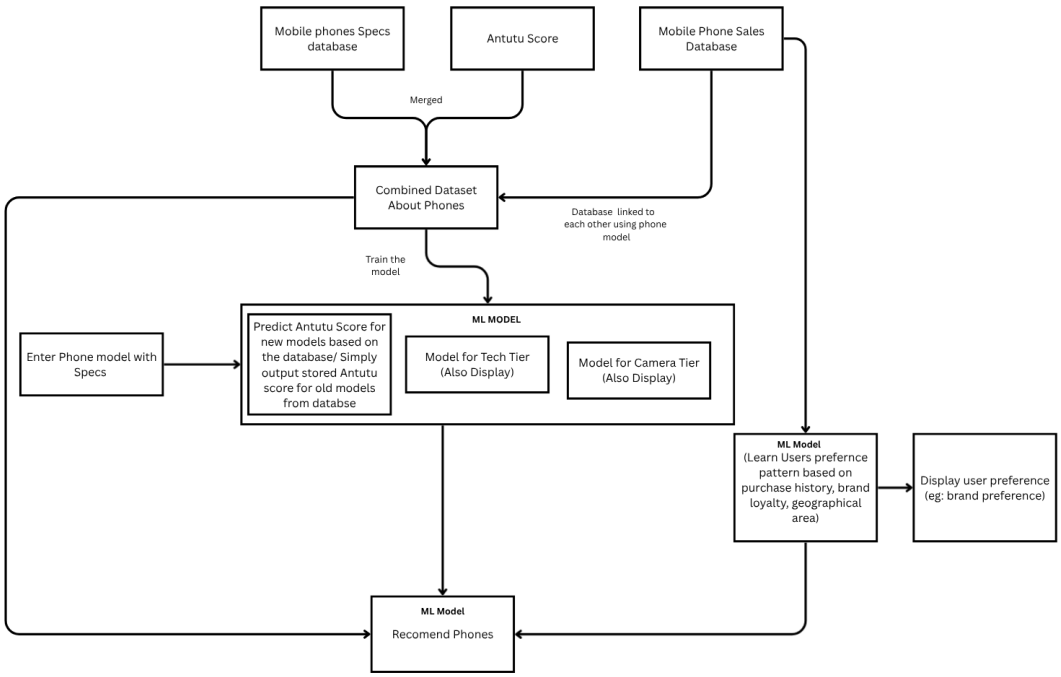

Figure VIII: Machine Learning Model Workflow

The machine learning workflow illustrates how the integrated dataset is used to train a unified XGBoost model. The mobile phone specifications database, Antutu scores, and mobile phone sales database are merged, creating a combined dataset about phones. This dataset is used to train the machine learning model, which learns to predict Antutu scores for new phone models based on their specifications. The model also performs tech tier classification, camera preference analysis, brand loyalty identification, and geographic brand recommendation. When a retailer enters a phone model with specifications, the system predicts its Antutu score and applies tech tier, camera, and brand logic to generate a complete audience profile. For customer facing recommendations, the system takes customer ID as input and recommends phones based on matching tech tier, camera preferences, and brand affinity, serving both retailer and customer needs

through an integrated pipeline.

3.5 Implementation Details

3.5.1 Frontend Development

The frontend is a React based single page application that talks to the backend through JSON HTTP requests. It is built for three types of users, which are customer, salesman, and admin, along with a shared dashboard that everyone can see. The customer side handles registration, six digit OTP verification, login, password reset, preference setup, and a list of recommended phones with full details. The salesman side has its own login and a dashboard where the user can search by location and see which phones are popular in that area, along with the camera and performance categories. The admin side has pages to view all users, add new users, update user details, remove users, and assign roles like customer, salesman, or admin. The shared dashboard shows summary cards at the top, a tech tier distribution panel, a camera preference distribution panel, and a recent activity feed, and right now the values are hard coded just to show the layout and will be connected to real data in the next phase. A toast component is also included so the app can show success and error messages, and the layout uses Bootstrap friendly classes so it looks good on both desktop and mobile screens.

3.5.2 Backend Development

The backend is built with Node.js and the Express.js framework, which together handle HTTP requests, routing, sessions, and authentication. The database is PostgreSQL and it is accessed through Prisma, which keeps the schema clean and handles migrations automatically. For login, the app uses Passport Local with bcrypt to hash passwords, sessions are stored in the same PostgreSQL database through connect-pg-simple so they survive restarts, and OTPs are generated with the crypto module and sent by email through Nodemailer. The routes are split based on who is using them. The customer routes cover registration, OTP verification, OTP resend, login, logout, and password reset. The salesman routes handle location based prediction requests and return the popular phones along with their camera and performance categories. The admin routes let the admin list, create, update, and delete users and also assign roles, and these routes are only available to users with the admin role. There is also a shared dashboard route that returns the aggregated statistics, tech tier and camera distributions, and recent activity for the frontend. A middleware called isAuthenticate protects routes that need a logged in user, express-validator checks the input through reusable schemas, and a central error handler returns clean JSON error messages. The project is organized into separate folders for routes, controllers, services, middleware, validation, configuration, and database access, and the Prisma schema is stored in prisma/schema.prisma with all migrations version controlled.

3.5.3 Machine Learning Model Integration

The machine learning part of the system is built with Python using XGBoost, pandas, and scikit-learn. The data preparation work, which includes scraping the AnTuTu benchmark scores and combining the three datasets, was done at the start of the project. Going forward, the system will train a set of XGBoost models on the combined dataset that includes phone specifications, AnTuTu scores, and mobile sales data, and all of this information is linked through the phone model name. When a salesman enters a phone model with its specifications, the model will predict the AnTuTu score and then figure out the tech tier, camera category, brand loyalty pattern, and the geographic area where the brand is popular, which together form a full audience profile. For customers, the system will take the customer ID as input and recommend phones that match their tech tier, camera preference, and brand affinity. The trained models will be saved as files using joblib or pickle and served to the Node.js backend through a small Python HTTP service, which the backend will call with timeouts and retries. Older versions of the models will be kept so that the team can roll back if a newer model does not perform as expected.

3.5.4 Database Schema

The database is PostgreSQL and it is managed through Prisma, with the schema saved in prisma/schema.prisma and all changes tracked through version controlled migrations. Right now the schema has two tables, which are users, which holds name, email, password hash, phone number, is active, and is verified, and otps, which holds the six digit code, expiry time, is used flag, and a foreign key back to the user with cascade delete so OTPs are removed when the user is removed. The schema will be extended later to add tables for phone specifications, customer preferences, recommendation history, brand loyalty, and location based brand statistics.

3.5.5 Testing Strategy

Unit Testing: Each part of the system is tested on its own. This covers password hashing, OTP generation, validation, Prisma queries, React components, and the model training script with a small dataset.

Integration Testing: The parts are tested together, like registration creating a user and an OTP in one step, login and logout handling the session, the protected route blocking users who are not logged in, and later the link between Node.js, the Python model service, and the database.

User Acceptance Testing: Real users try the app and give feedback. Salesmen test the location search and customers test registration, preferences, and recommendations.

Performance Testing: Response times for predictions and recommendations are mea-

sured to keep them under two seconds, the system is tested with many users at the same time, and behavior on different devices and browsers is checked.

4. Time Schedule

The Customer Segmentation and Intelligent Mobile Recommendation System project is scheduled according to the timeline illustrated in Figure **??**.

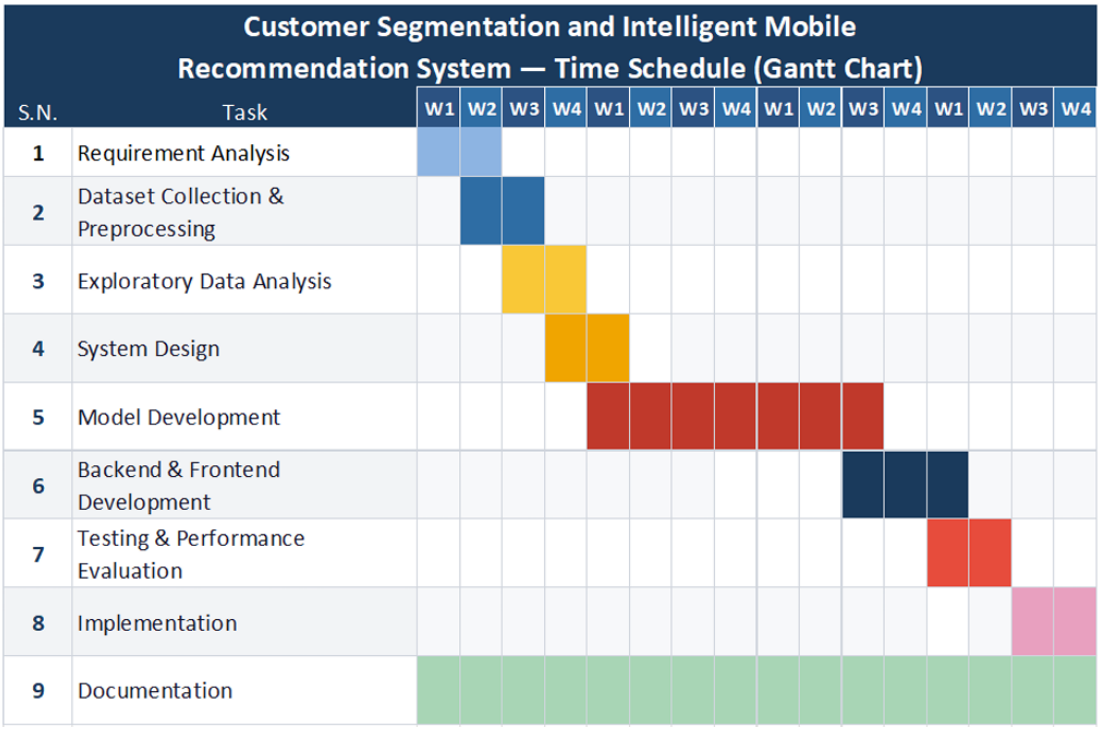

Figure I: Gantt chart for Customer Segmentation and intelligent Mobile Recommendation System

The project began with data collection and preprocessing in the first week, covering web scraping of AnTuTu benchmark scores and merging the three datasets. Requirement analysis, system design, and exploratory data analysis ran alongside this preparation. Backend and frontend development took place from the second week through the current phase, building the Node.js backend with Express.js and the React based web interface. The documentation has been done from the beginning of the project to the end.

Model development, XGBoost training, model serving, and the integration of the recommendation endpoints are scheduled for the next project phase. Testing and performance evaluation will follow, together with final implementation, deployment, and any remaining documentation work.

5. Expected Output

This chapter is divided into two parts. The first part lists the work that has been completed by the mid term defense and is already visible in the current code repository. The second part lists the work that is still in progress and is planned for the next project phase.

5.1 Work Completed

5.1.1 Frontend

The frontend of the system has been developed using React. It includes a sign in page, a multi step registration page with OTP verification, and a password reset page. Client side validation, loading indicators, and error messages have been added to improve the user experience.

A reusable OTP input component was created with automatic focus movement, paste support, backspace navigation, a resend timer, and inline validation. A reusable toast notification component was also implemented to display success and error messages from the backend.

A dashboard interface has been designed with summary cards, charts, and a recent activity section. The dashboard currently uses hard coded data and is prepared for integration with backend APIs.

5.1.2 Backend

The backend has been built using Node.js and Express.js with PostgreSQL managed through Prisma ORM. The database schema, migrations, and relationships between users and OTP records have been completed to support the authentication system.

A complete authentication module has been implemented, including user registration, email based OTP verification, login, logout, password reset, and session management. Passwords are securely hashed, user input is validated, and sessions are stored in PostgreSQL for persistence.

Additional backend features include centralized error handling, reusable validation middleware, protected routes, and basic CRUD operations for users. These provide a solid foundation for future application development.

5.1.3 Machine Learning

The required datasets have been collected, cleaned, and preprocessed. Smartphone specifications, benchmark scores, and mobile sales data were merged into a single

dataset to prepare it for machine learning model training.

The processed dataset is now ready for developing prediction and recommendation models. This data preparation forms the foundation for the intelligent recommendation system that will be integrated into the application.

5.2 Work in Progress

5.2.1 Frontend

The frontend dashboard is being connected to live backend APIs so that the current hard coded data can be replaced with real time information. Phone recommendation cards with match scores, camera tiers, and recommendation reasons are also being developed.

An admin dashboard for user management and system monitoring is under development. Accessibility improvements are also being added, including better focus indicators, image descriptions, and improved colour contrast.

5.2.2 Backend

The backend is being extended with new recommendation APIs and additional database tables to store phone information, customer preferences, recommendations, and purchase history. Role based access control is also being added to separate retailer and customer features.

Further improvements include rate limiting for authentication, structured logging, comprehensive testing, Docker support, GitHub Actions for automation, performance optimization, caching, and updated project documentation.

5.2.3 Machine Learning

The machine learning models are currently being trained using the merged dataset. Multiple XGBoost models are being developed for score prediction, customer segmentation, camera preference, technology tier classification, and phone recommendation.

A lightweight Python prediction service is also being built to load the trained models and provide prediction results through an API. This service will communicate with the Node.js backend to generate intelligent phone recommendations.

References

[1] Bhavesh Singh. Customer segmentation recommendation. https://github.com/bhaveshsingh0206/customer-segmentation-recommendation, 2020. Accessed: 2026-02-26.

[2] Brij Kishore Pandey. The cold start problem in recommender systems, 2022. URL https://www.freecodecamp.org/news/cold-start-problem-in-recommender-systems/. Accessed: 2024-05-20.

[3] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[4] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.

[5] Peng Qi, Xiaolong Zhang, and Zhenyu Wang. An xgboost recommendation algorithm based on collaborative filtering. In *2018 IEEE 9th International Conference on Software Engineering and Service Science (ICSESS)*, pages 138–141, 2018. doi: 10.1109/ICSESS.2018.8663849.

[6] Kavitha Ayyappan et al. Bidirectional lstm with xgboost for customer purchase intent prediction. *International Journal of Intelligent Systems and Applications in Engineering*, 11(4):284–293, 2023.

[7] dilkushsingh. Smartphones dataset (august 2024). https://www.kaggle.com/datasets/dilkushsingh/smartphones-dataset-upto-july24, 2024. Accessed: 2026-02-27.

[8] waqi786. Mobile sales dataset. https://www.kaggle.com/datasets/waqi786/mobile-sales-dataset, 2024. Accessed: 2026-02-27.

[9] NanoReview. Phone list by antutu rating. https://nanoreview.net/en/phone-list/antutu-rating, 2026. Accessed: 2026-02-27.